MACHINE LEARNING WITH SCIKIT-LEARN

# Unsupervised Learning + Hyperparameter Tuning

07

Find structure nobody labeled — clusters, compressed views — then squeeze your best model with Grid & Randomized search.

**Instructor: Shutchon Premchaisawatt**

Master's Degree Program · Hands-on with Google Colab

AGENDA

# Today's agenda

01 **Unsupervised: no labels**
what can we still learn?

02 **KMeans**
centroids & the assign-update loop

03 **Choosing k**
elbow & silhouette

04 **Blind experiment**
wine: rediscover 3 cultivars

05 **DBSCAN**
density, noise, weird shapes

06 **PCA**
compress 13 features → 2D picture

07 **Grid & Randomized search**
tuning without self-deception

08 **Learning curves**
more data or more model?

NEW WORLD

# What if nobody gives you y?

### Clustering

Group similar rows.

Customer segments · patient phenotypes · document topics

→ KMeans, DBSCAN

### Dimensionality reduction

Compress many columns into few.

Visualize 13-D data · de-noise · speed up downstream models

→ PCA

### Anomaly detection

Find the weird ones.

Fraud probes · sensor faults · DBSCAN noise points

(comes free with clustering)

- **No score without y — evaluation becomes:**
- internal (silhouette) · stability · human judgment · proxy tasks

# KMeans: plant flags, regroup, repeat

**1**
① **pick k centroids**
random start

→

**2**
② **assign**
each point → nearest centroid

→

**3**
③ **update**
centroid → mean of its group

→

**4**
④ **repeat ②③**
until stable

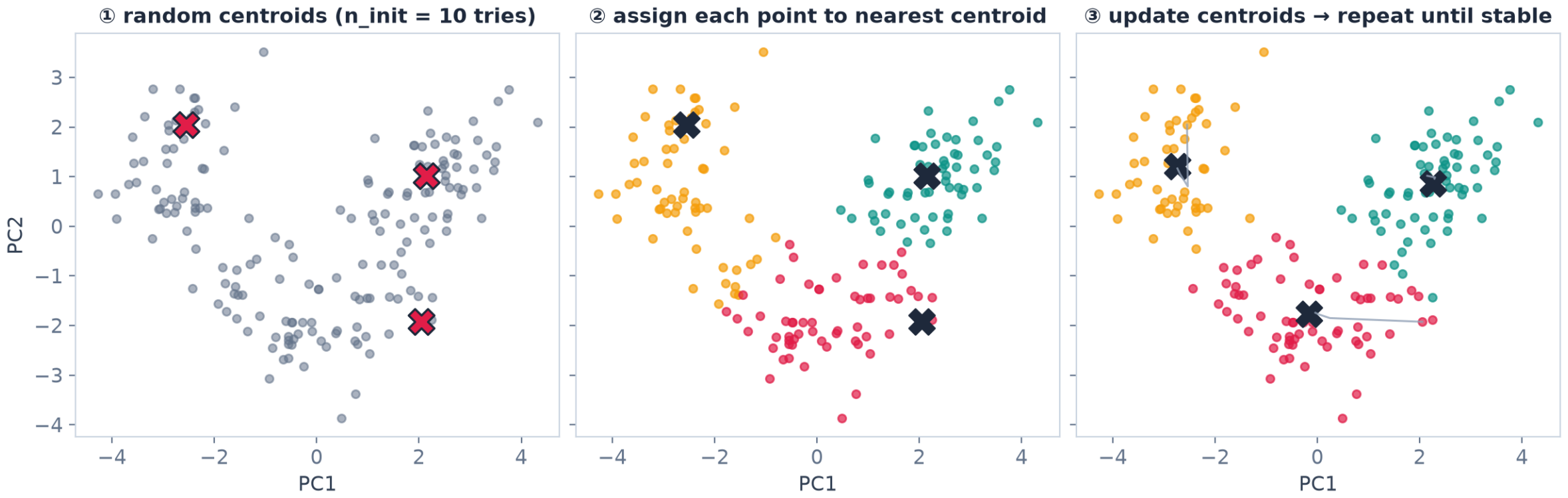
```
from sklearn.cluster import KMeans
km = KMeans(n_clusters=3, n_init=10, random_state=42)
labels = km.fit_predict(X_scaled)      # X has NO y
```

- **inertia_ = total squared distance to centroids (lower = tighter) · labels_ = cluster id per row (0,1,2…)**
- **Scaling FIRST — distances again (S03!)**

## Assumes

round-ish, similar-size clusters.
Struggles on moons, rings, unequal blobs.
n_init=10 = retry 10 starts, keep best (sklearn ≥1.4 default).

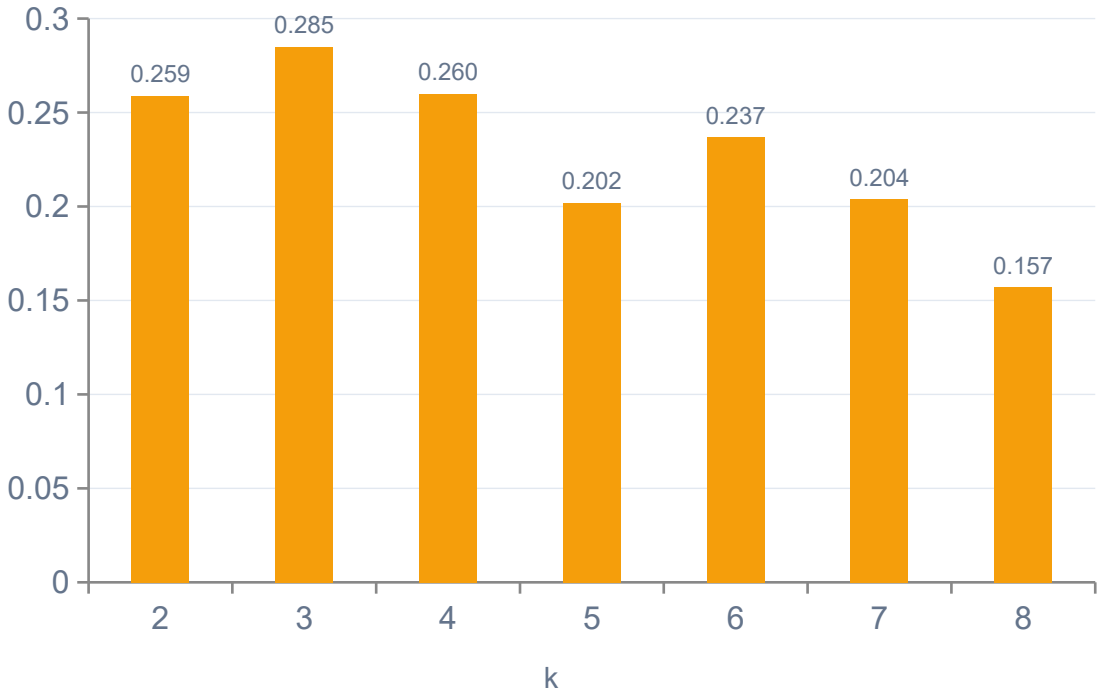
# KMeans in pictures — assign & update




- Real fit on wine (PCA-2D view): random centroids → assign every point to its nearest → move each centroid to its group's mean → repeat until stable
- Always converges — but only to a LOCAL optimum. That's exactly why n_init=10 retries ten different starts and keeps the best.

# Elbow & silhouette (wine, 13 features, scaled)




- **Elbow: inertia always falls — look for the bend (≈3)**
- **Silhouette = cohesion vs separation, −1…1; peaks at k=3 (0.285)**
- Both are guides, not verdicts — finally a HUMAN judges if segments make business sense

★ THE REVEAL

# Wine: KMeans found the 3 cultivars — blind

178 Italian wines · 13 chemical features · labels (3 cultivars) HIDDEN from the algorithm

**98.3%**

agreement with the hidden cultivar labels (best mapping)

**0.90**

adjusted Rand index (chance-corrected match)

**65/51/62**

cluster sizes found (truth: 59/71/48)

**Structure was IN the chemistry. No teacher, no labels — KMeans recovered the taxonomy. That's what "unsupervised" buys you.**

AFTER CLUSTERING

# Clusters are only useful if you can NAME them

```
df["cluster"] = labels

# how big is each segment?
df["cluster"].value_counts().sort_index()

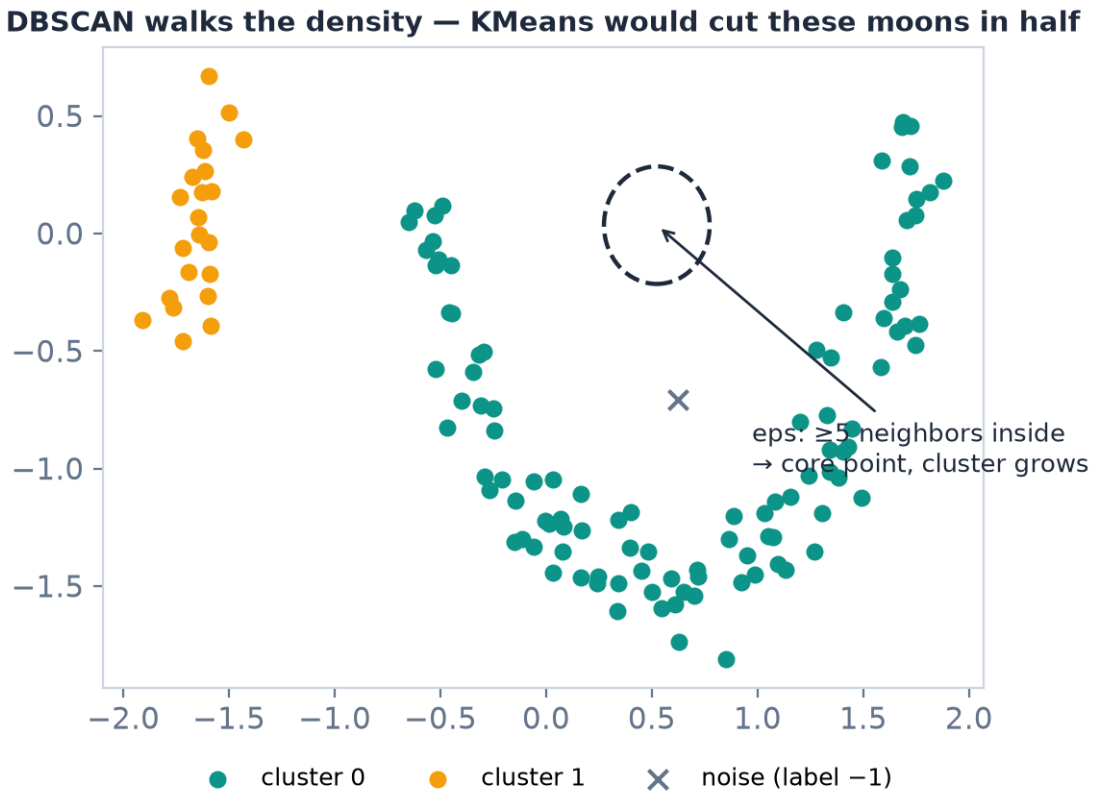
# profile each cluster on original features
df.groupby("cluster").mean().round(1)

# the segment-naming moment:
df.groupby("cluster")[["alcohol", "flavanoids"]].mean()

# export the segments for the business team
df.to_csv("wine_segments.csv", index=False)
```

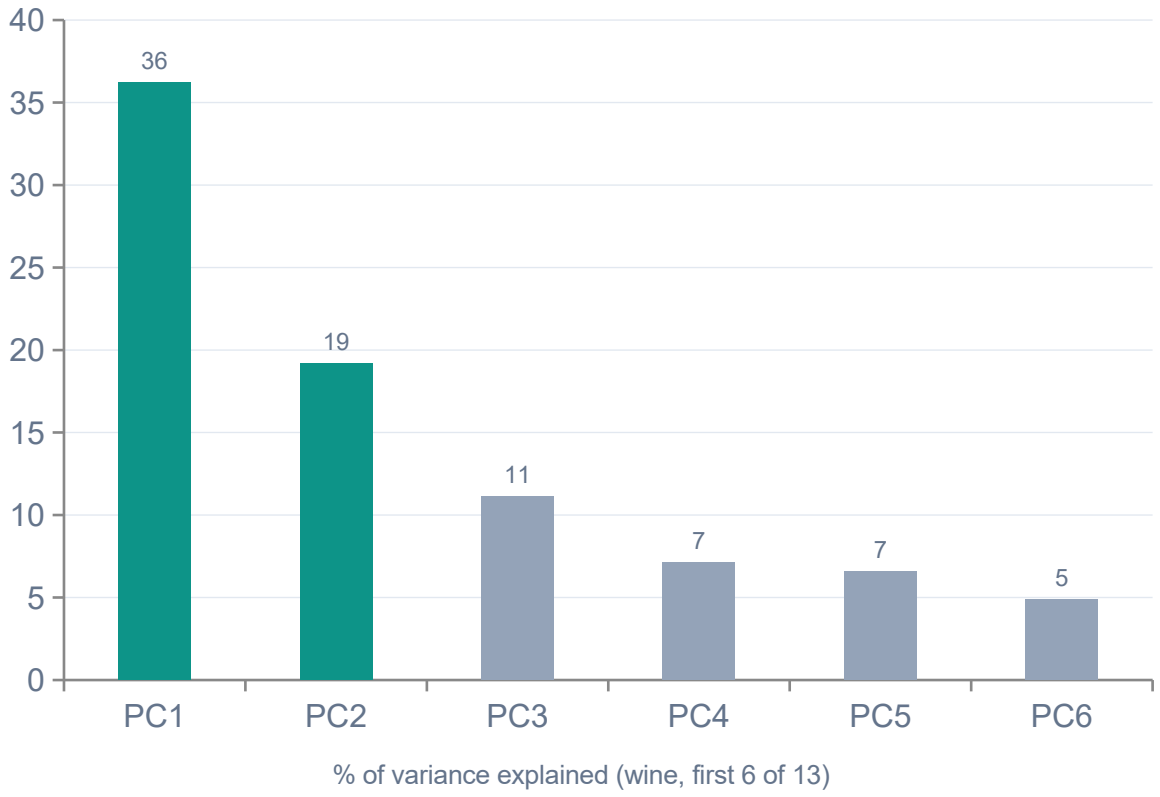
- **The analyst's job**
- read the profiles → invent names:
- "low-alcohol/high-phenol group" …
- **For business data this is where money is made**
- "Segment 2: high income, low spending → target them"
  - Anomaly bonus: rows far from any centroid = unusual

# DBSCAN: follow the density, flag the weird




- **How it works**
- a cluster = region dense enough (min_samples within eps)
- no k to guess — density decides
- **Superpowers**
- arbitrary shapes · automatic outlier flagging (label −1)
- **Knobs: eps, min_samples — sensitive; use a k-distance plot**
- **KMeans = round blobs, fast, scalable · DBSCAN = shapes + noise, fussier**

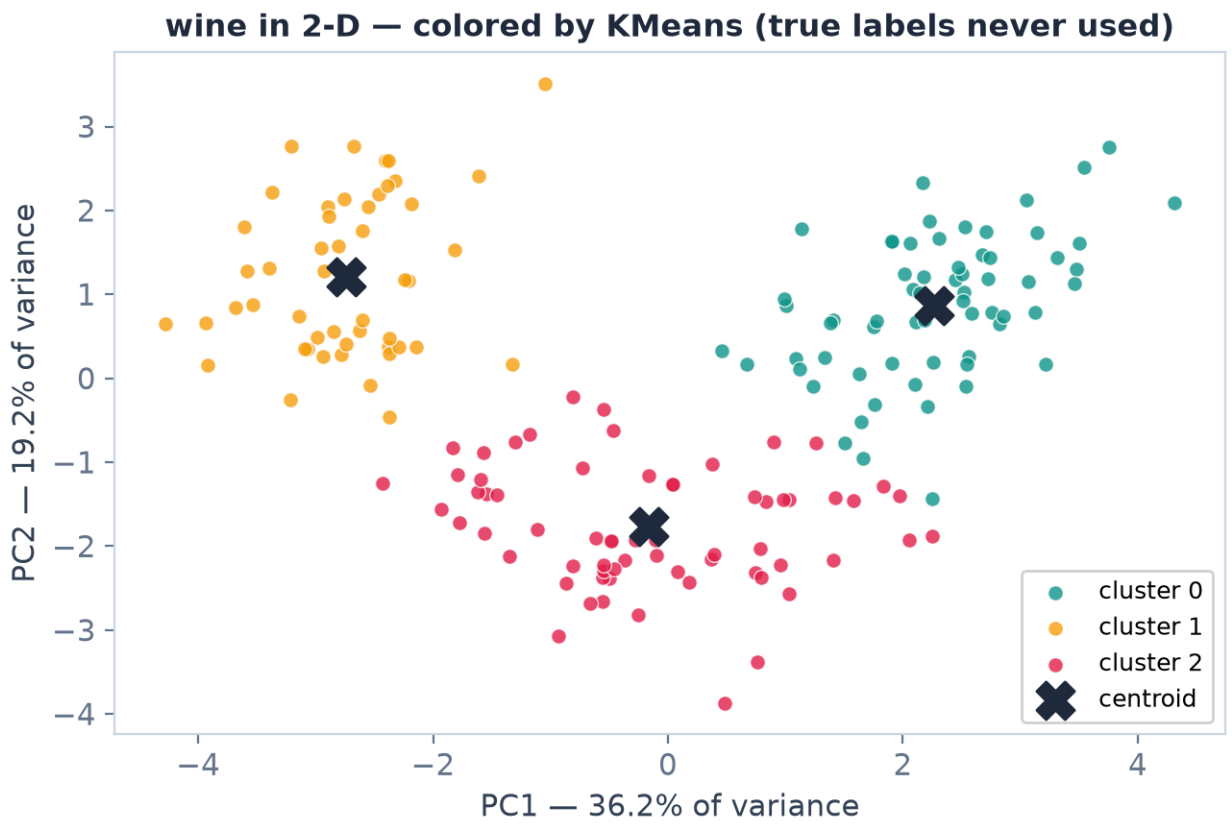
# PCA: 13 features → 2 axes that matter most




- **Idea**
- find the directions where the data varies most
- keep the top few, drop the rest
- **Wine result (scaled):**
- PC1 explains 36.2% · PC2 19.2%
- 2 axes = 55.4% of 13 columns' story
- 10 of 13 needed for 95% — real data is spread out
- **Uses**
- plot 13-D data as a 2-D scatter (next slide)
- de-noise / speed up pipelines
- **Always scale first — PCA hunts VARIANCE, and fare… er, proline has the biggest raw scale**

THE PICTURE

# PCA 2-D scatter: clusters you can SEE




```
Z = PCA(n_components=2).fit_transform(X_scaled)
sns.scatterplot(x=Z[:,0], y=Z[:,1], hue=labels)
```

- **3 clouds = 3 cultivars, visible**
- plot colored by cluster OR by true label — same picture (ARI 0.90!)
- **This chart is also your unsupervised sanity check**

PART 2

# Now squeeze: hyperparameter tuning

| model | hyperparameters (knobs) | who sets them |
|---|---|---|
| KNN | n_neighbors, weights | YOU — before fit() |
| DecisionTree | max_depth, min_samples_leaf | YOU |
| RandomForest | n_estimators, max_features, … | YOU |
| Ridge / Lasso | alpha | YOU |
| DBSCAN | eps, min_samples | YOU |
| Linear regression | (none — parameters w, b are learned) | the data |

**Defaults are good, not optimal. Tuning = search knobs with CV — never with the test set.**

# GridSearchCV: try every cell of the grid




2 × 3 = 6 candidates × 5 CV folds = 30 fits — sklearn runs them all

```
gs = GridSearchCV(model, {"n_estimators": [100, 300],
                          "max_depth": [None, 6, 4]}, cv=5)
gs.fit(X_train, y_train)   # best_params_, best_score_ inside
```

- **It's the S04 protocol, automated**
- every candidate judged by the same CV
- **best_score_ = CV score of the winner**
  - Re-fit on ALL training data happens automatically

# RandomizedSearchCV: when the grid explodes

- **Grid pain**
- 6 hyperparameters × 5 values each = 15,625 combos × 5 folds
- **Random fix**
- sample n_iter=30 random points from DISTRIBUTIONS
- n_estimators: randint(50, 500) — any integer, not a fixed list
- **30 good guesses ≈ 15,625 exhaustive ones — usually**

```
from scipy.stats import randint
rs = RandomizedSearchCV(
    RandomForestClassifier(random_state=42),
    {"n_estimators": randint(50, 500),
     "max_depth": [None, 4, 6, 8, 12],
     "min_samples_leaf": [1, 2, 4],
     "max_features": ["sqrt", "log2", None]},
    n_iter=30, cv=5, random_state=42)
```

```
best CV 0.965: depth 8 · log2 · 97 trees · leaf 1
```

HONESTY CHECK

# Tuning worked… then didn't (and that's informative)

| RandomForest (breast cancer) | CV | test |
|---|---|---|
| default knobs | — | 0.958 |
| after RandomizedSearch (30×5) | 0.965 | 0.958 |

- **CV improved (+0.7pt), test didn't move**
- this dataset is near its ceiling — the signal separates cleanly
- **Tuning pays big when: the search space is rich · defaults are far off · the problem is harder (S08 churn!)**
- **The PROCESS is still correct — report both numbers honestly**

### Reflex to avoid

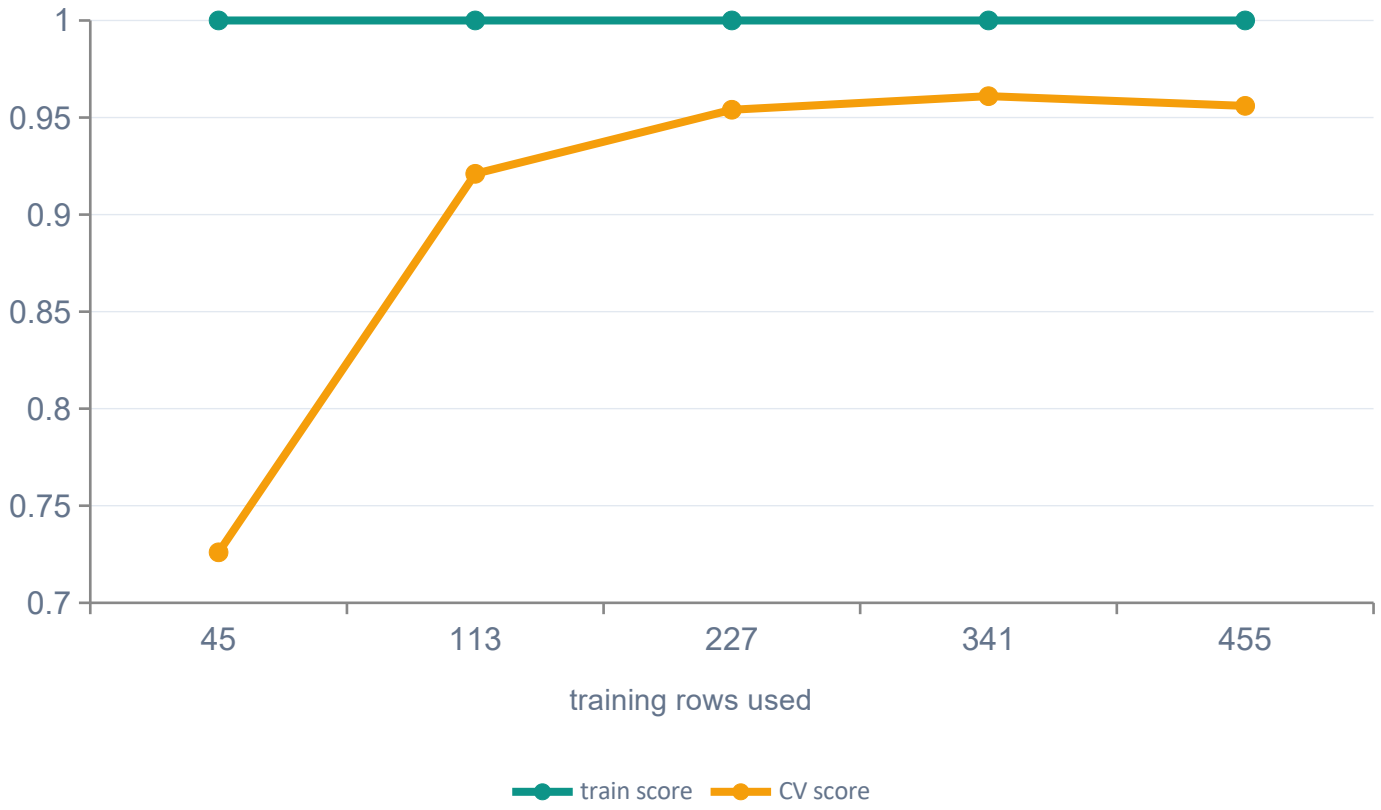
Tuning until the TEST number finally rises = tuning on the test set. Soft leakage (S04).

### Beyond today

HalvingRandomSearchCV — successive halving, smarter budget on big data.

DIAGNOSIS

# Learning curves: more data or more model?




- **Read it**
- CV climbs 0.73 → 0.96 as rows grow: DATA was the bottleneck
- train pinned at 1.0: RF memorizes any training set
- **Converging curves ≈ plateau — more rows won't help much now**
- learning_curve(model, X, y, cv=5, train_sizes=…)

DIAGNOSIS

# Symptom → prescription

| symptom | diagnosis | prescription |
|---|---|---|
| train ↑ / test ↓ (gap) | overfitting | more regularization, simpler model, more data, fewer features |
| both low, small gap | underfitting | richer model, more features, less regularization |
| CV unstable across folds | high variance | more data, simpler model, CV with more folds |
| CV ≫ test | tuned to CV / shift | trust test; simpler search; check leakage again |

SHIPPING

# Save the winner — and the reproducibility kit

```
import joblib
joblib.dump(best_pipeline, "churn_model.joblib")

clf = joblib.load("churn_model.joblib")   # anywhere
clf.predict(new_raw_rows)                 # transforms inside
```

- **Dump the PIPELINE, not just the model**
- preprocessor + model travel together
- **Reproducibility kit**
- random_state everywhere · library versions ·
- the exact CV object · who ran what, when
- **S08 project deliverable = exactly this kit**

## Final pitfall checklist

split first · transform inside · WHEN known? · duplicates? · same folds for comparisons · right metric · Dummy anchor · test ONCE

LAB 07 · DO IT IN GOOGLE COLAB

# ค้นพบกลุ่มที่ซ่อนอยู่ + ปรับแต่งโมเดล — wine & tuning

**YOUR MISSION (โจทย์ภาษาไทยใน Colab)**

- โหลด wine (13 features) แล้ว scale — รัน KMeans k=2..8 เก็บ inertia และ silhouette — k เท่าไหร่ดีที่สุด?
- เฉลย: เทียบ labels ที่ได้กับ y จริงด้วย adjusted_rand_score — คอมเมนต์ผล
- วาด PCA 2 แกน ระบายสีด้วย cluster — เห็น 3 กลุ่มไหม อธิบาย explained variance
- profile แต่ละ cluster ด้วย groupby().mean() แล้วตั้งชื่อกลุ่มเป็นภาษาไทย
- ปรับแต่ง: RandomizedSearchCV บน RandomForest ของ breast cancer — เทียบ CV ก่อน/หลัง และ test ครั้งเดียว

## Learning goal

Unsupervised discovery on wine (elbow/silhouette, blind validation, PCA picture) plus a full tune of your best classifier.

## Vibe coding corner

ลองถาม AI: “ช่วยออกแบบการทดลอง tune RandomForest ให้ครอบคลุมแต่ไม่แพงเกิน 30 fits” แล้วเช็กว่ามันเสนออะไรเกินจำเป็น/ผิดหลัก leakage ไหม

## KEY TAKEAWAYS

1. Unsupervised finds structure without labels — KMeans on wine recovered the 3 cultivars at 98% agreement.
2. Choose k with elbow + silhouette, then make clusters useful by NAMING them (profile → segment story).
3. DBSCAN for shapes + noise; PCA to compress & visualize (13 features → 2 axes = 55% of wine's story).
4. Tune with Grid/RandomizedSearchCV on CV — 30 random guesses often match a huge grid.
5. Honesty: CV 0.965 → test 0.958 happens; report both. Dump the whole pipeline with joblib.

## NEXT

Session 08 — Final Project — Customer Churn Prediction, end to end